

ALGORITMA DAN STRUKTUR DATA
LATIHAN PRAKTIKUM 6 REKURSIF



Dosen Pembimbing:

Prof. Dr Arna Fariza S.Kom., M.Kom

Disusun oleh:

Nama: Luthfi Bahrur Rozaq

NRP: 3125600069

1 D4 TEKNIK INFORMATIKA C
PROGRAM STUDI TEKNIK INFORMATIKA
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA

TUGAS PRAKTIKUM ASD – REKURSIF

1. Buatlah program rekursif untuk menghitung segitiga Pascal !

```
F1          1
F2         1 1
F3        1 2 1
F4       1 3 3 1
F5      1 4 6 4 1
F6     1 5 10 10 5 1
```

Kode Program:

```
1. #include <stdio.h>
2.
3. int pascal(int baris, int kolom);
4.
5. int main() {
6.     int baris;
7.     printf("masukkan jumlah baris segitiga pascal (misal: 6):
8.     ");
9.     scanf("%d", &baris);
10.
11.     for (int i = 0; i < baris; i++) {
12.         for(int space = 0; space < baris - i; space++) {
13.             printf(" ");
14.         }
15.         for (int j = 0; j <= i; j++) {
16.             printf("%4d", pascal(i, j));
17.         }
18.         printf("\n");
19.     }
20.     return 0;
21.
22. int pascal(int baris, int kolom) {
23.     if (kolom == 0 || kolom == baris) {
24.         return 1;
25.     } else {
26.         return pascal(baris - 1, kolom - 1) + pascal(baris -
27.         1, kolom);
28.     }
```

Output:

masukkan jumlah baris segitiga pascal (misal: 6): 7

```
      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
1 6 15 20 15 6 1
```

Analisis Program:

Program ini mendemonstrasikan implementasi dasar dari relasi rekurens matematis dalam membentuk Segitiga Pascal, di mana algoritma bekerja dengan memecah perhitungan nilai suatu sel menjadi proses penjumlahan dua sel yang berada tepat di baris atasnya. Alur program ini secara presisi menangani skenario khusus (basis/stop case) pada kondisi tepi kolom, yakni saat indeks kolom bernilai nol atau sama dengan indeks baris, untuk mengembalikan nilai absolut satu, sehingga tumpukan pemanggilan fungsi di dalam memori (call stack) dapat dievaluasi kembali ke atas tanpa menyebabkan pengulangan eksekusi tak hingga (infinite loop) yang membebani sistem.

2. Buatlah program secara rekursif, masukkan jumlah N karakter dan cetak dalam semua kombinasi !

Jumlah karakter = 3

aaa aab aac aba abb abc aca acb acc baa bab bac bba bbb bbc bca
bcb bcc caa cab cac cba cbb cbc cca ccb ccc BUILD SUCCESSFUL
(total time: 1 second)

```
1. #include <stdio.h>
2.
3. void kombinasi(char arr[], int n, int panjang, int index);
4.
5. int main() {
6.     int n;
7.     printf("jumlah karakter = ");
8.     scanf("%d", &n);
9.
10.    char arr[10];
11.    kombinasi(arr, n, n, 0);
12.
13.    printf("\nbuild successful\n");
14.    return 0;
15.}
16.
17. void kombinasi(char arr[], int n, int panjang, int index) {
```

```

18.     if (index == panjang) {
19.         for (int i = 0; i < panjang; i++) {
20.             printf("%c", arr[i]);
21.         }
22.         printf(" ");
23.         return;
24.     }
25.
26.     for (int i = 0; i < n; i++) {
27.         arr[index] = 'a' + i;
28.         kombinasi(arr, n, panjang, index + 1);
29.     }
30. }

```

Output:

```

jumlah karakter = 3
aaa aab aac aba abb abc aca acb acc baa bab bac bba bbb bbc bc
a bcb bcc caa cab cac cba cbb cbc cca ccb ccc
build successful

```

Analisis Program:

Program ini mendemonstrasikan eksplorasi memori secara rekursif untuk menghasilkan kombinasi susunan karakter secara terurut, di mana algoritma memanipulasi sebuah array penampung sementara menggunakan pointer penunjuk indeks batas. Operasi ini menelusuri setiap probabilitas cabang karakter mulai dari huruf awal hingga mencapai batas kedalaman maksimum yang ditetapkan sebagai stop case, untuk kemudian mencetak isi dari susunan memori saat itu dan melakukan proses runut balik (backtracking) secara presisi ke cabang sebelumnya hingga seluruh kombinasi berhasil dieksekusi tanpa melewatkan satupun susunan.

3. Buat program BinarySearch dengan Rekursif ! (data tentukan sendiri)
 Data : 2,5,8,10,14,32, 35, 41, 67, 88, 90, 101, 109
 Data yang dicari : 10
 Data 10 berada pada indek ke - 3

```

1. #include <stdio.h>
2.
3. int binarysearch(int arr[], int l, int r, int x);
4.
5. int main() {
6.     int data[] = {2, 5, 8, 10, 14, 32, 35, 41, 67, 88, 90,
7.                   101, 109};
8.     int n = sizeof(data) / sizeof(data[0]);
9.     int cari = 10;

```

```

9.
10.     printf("data: ");
11.     for(int i = 0; i < n; i++) printf("%d ", data[i]);
12.     printf("\ndata yang dicari: %d\n", cari);
13.
14.     int hasil = binarysearch(data, 0, n - 1, cari);
15.
16.     if (hasil == -1)
17.         printf("data tidak ditemukan di dalam array.\n");
18.     else
19.         printf("data %d berada pada indek ke\n%d\n", cari,
    hasil);
20.
21.     return 0;
22. }
23.
24. int binarysearch(int arr[], int l, int r, int x) {
25.     if (r >= l) {
26.         int mid = l + (r - l) / 2;
27.
28.         if (arr[mid] == x)
29.             return mid;
30.
31.         if (arr[mid] > x)
32.             return binarysearch(arr, l, mid - 1, x);
33.
34.         return binarysearch(arr, mid + 1, r, x);
35.     }
36.
37.     return -1;
38. }

```

Output:

```

data: 2 5 8 10 14 32 35 41 67 88 90 101 109
data yang dicari: 10
data 10 berada pada indek ke
3
data: 2 5 8 10 14 32 35 41 67 88 90 101 109
data yang dicari: 35
data 35 berada pada indek ke
6

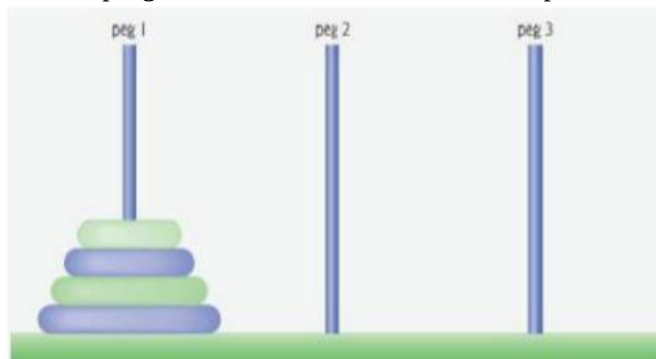
```

```
data: 2 5 8 10 14 32 35 41 67 88 90 101 109
data yang dicari: 100
data tidak ditemukan di dalam array.
```

Analisis Program:

Program ini mendemonstrasikan tingkat efisiensi komputasi yang tinggi menggunakan metode pencarian divide and conquer berbasis rekursi yang bekerja secara eksklusif pada tatanan data terurut. Algoritma mengeksekusi penelusuran dengan membelah ruang array menjadi dua bagian secara iteratif pada setiap pemanggilan fungsinya, menggunakan perhitungan titik tengah (midpoint) untuk mengeliminasi separuh blok data yang tidak relevan, hingga akhirnya node target tervalidasi pada indeks yang tepat, atau tumpukan parameter batas kiri menyilang melewati batas kanan yang bertindak sebagai pemutus rekursi saat data dipastikan tidak eksis di dalam sistem.

4. Buatlah program rekursif untuk memecahkan permasalahan Menara Hanoi !



Program ini merupakan program untuk menampilkan pergerakan menara hanoi, yang merujuk pada class menaraHanoi. Secara umum algoritma menara hanoi, adalah memindahkan sub menara hanoi dengan $n - 1$ pin dari n pin ke tiang perantara. Lalu memindahkan pin ke n ke tiang tujuan, lalu memindahkan sub menara hanoi dengan $n - 1$ pin yang ada di tiang perantara, ke tiang tujuan. StopCase nya jika $n == 1$.

Jumlah disk : 3 Langkah-langkah nya adalah dengan :

1. Pindahkan disc 1 dari pasak A ke pasak C
2. Pindahkan disc 2 dari pasak A ke pasak B
3. Pindahkan disc 1 dari pasak C ke pasak B
4. Pindahkan disc 3 dari pasak A ke pasak C
5. Pindahkan disc 1 dari pasak B ke pasak A
6. Pindahkan disc 2 dari pasak B ke pasak C
7. Pindahkan disc 1 dari pasak A ke pasak C

```
1. #include <stdio.h>
2.
3. void menarahanoi(int n, char asal, char tujuan, char
   perantara);
4.
5. int main() {
```

```

6.     int n = 3;
7.     printf("jumlah disk: %d\n", n);
8.     printf("langkah-langkah nya adalah dengan :\n");
9.
10.    menarahanoi(n, 'a', 'c', 'b');
11.
12.    return 0;
13.}
14.
15.void menarahanoi(int n, char asal, char tujuan, char
    perantara) {
16.    if (n == 1) {
17.        printf("pindahkan disc 1 dari pasak %c ke pasak
            %c\n", asal, tujuan);
18.        return;
19.    }
20.
21.    menarahanoi(n - 1, asal, perantara, tujuan);
22.
23.    printf("pindahkan disc %d dari pasak %c ke pasak %c\n",
        n, asal, tujuan);
24.
25.    menarahanoi(n - 1, perantara, tujuan, asal);
26.}

```

Output:

```

jumlah disk: 3
langkah-langkah nya adalah dengan :
pindahkan disc 1 dari pasak a ke pasak c
pindahkan disc 2 dari pasak a ke pasak b
pindahkan disc 1 dari pasak c ke pasak b
pindahkan disc 3 dari pasak a ke pasak c
pindahkan disc 1 dari pasak b ke pasak a
pindahkan disc 2 dari pasak b ke pasak c
pindahkan disc 1 dari pasak a ke pasak c

```

```
jumlah disk: 4
langkah-langkah nya adalah dengan :
pindahkan disc 1 dari pasak a ke pasak b
pindahkan disc 2 dari pasak a ke pasak c
pindahkan disc 1 dari pasak b ke pasak c
pindahkan disc 3 dari pasak a ke pasak b
pindahkan disc 1 dari pasak c ke pasak a
pindahkan disc 2 dari pasak c ke pasak b
pindahkan disc 1 dari pasak a ke pasak b
pindahkan disc 4 dari pasak a ke pasak c
pindahkan disc 1 dari pasak b ke pasak c
pindahkan disc 2 dari pasak b ke pasak a
pindahkan disc 1 dari pasak c ke pasak a
pindahkan disc 3 dari pasak b ke pasak c
pindahkan disc 1 dari pasak a ke pasak b
pindahkan disc 2 dari pasak a ke pasak c
pindahkan disc 1 dari pasak b ke pasak c
```

Analisis Program:

Program ini mendemonstrasikan pemecahan masalah manipulasi perpindahan berantai pada Menara Hanoi dengan alur logika rekursi yang secara elegan memetakan pemindahan piringan antar pasak. Algoritma beroperasi dengan mendelegasikan instruksi besar menjadi sub-masalah identik, di mana program menyisihkan $n-1$ piringan ke tiang perantara (sebagai pointer bantuan), memindahkan piringan target secara mutlak ke tiang tujuan, lalu memanipulasi kembali sisa piringan ke arah tujuan dengan stop case yang secara ketat memberhentikan pemecahan saat hanya tersisa satu piringan tunggal agar alur perpindahan rantai piringan tetap logis, aman, dan berurutan.

Kesimpulan

Pelaksanaan praktikum Algoritma dan Struktur Data mengenai fungsi Rekursif ini secara komprehensif membuktikan bahwa arsitektur pemanggilan sub-program ke dirinya sendiri mampu menawarkan solusi penulisan algoritma yang jauh lebih ringkas, elegan, dan terstruktur dibandingkan perulangan iteratif biasa (seperti for atau while), terutama untuk permasalahan yang memiliki pola pemecahan teratur. Pengelolaannya menuntut ketelitian analitis yang tinggi dalam mendefinisikan rumusan relasi rekurens dan kewajiban menyisipkan kondisi khusus (stop case) sebagai penahan absolut agar tumpukan pemanggilan fungsi (call stack) tidak merusak alokasi memori sistem (stack overflow) saat mengeksekusi instruksi yang bersarang.

Struktur dasar logika rekursif ini juga menunjukkan keandalannya saat diterapkan untuk memecahkan masalah komputasi kompleks yang membutuhkan pembelahan area kerja, seperti fondasi pencarian pada algoritma Binary Search yang efisien dalam memangkas waktu proses. Hingga pada akhirnya algoritma ini berhasil diekspansi menjadi purwarupa solusi matematis yang sanggup mengurai simulasi manipulasi pemindahan berantai pada Menara Hanoi maupun pemetaan pencabangan kombinasi karakter secara presisi tanpa perlu menyusun jangkar perulangan manual yang rumit dan berpotensi merusak alur logika arsitektur program secara keseluruhan.